1. Register Transfer Operations with Arithmetic and Logic Operations

AIM

To design and simulate a system that performs register transfer operations and basic arithmetic and logic operations such as addition, subtraction, AND, and OR using multiple registers and an ALU.

PROCEDURE

1. Initialize registers R1, R2, R3, and R4 with suitable data values.
2. Transfer the contents of R1 to another register using a register transfer operation.
3. Transfer the contents of R2 to the ALU.
4. Perform addition of the contents of R1 and R2 using the ALU and store the result in R3.
5. Perform subtraction of R2 from R1 and store the result in R4.
6. Perform AND operation between R1 and R2.
7. Perform OR operation between R1 and R2.
8. Observe the contents of the registers after every operation.
9. Analyze the role of the ALU in performing arithmetic and logical operations.

INPUT

Register	Data
R1	25
R2	10

REGISTER TRANSFER OPERATIONS

$R3 \leftarrow R1$

$R4 \leftarrow R2$

ALU OPERATIONS

$R3 \leftarrow R1 + R2$

$R4 \leftarrow R1 - R2$

$R3 \leftarrow R1$ AND $R2$

$R4 \leftarrow R1$ OR $R2$

OUTPUT

Operation	Result
$R3 \leftarrow R1$	25
$R4 \leftarrow R2$	10

R1 + R2	35
R1 − R2	15
R1 AND R2	8
R1 OR R2	27

ANALYSIS

The registers temporarily store the data required for processing. The control unit generates the required control signals for transferring data between registers and the ALU. The ALU performs arithmetic operations such as addition and subtraction and logical operations such as AND and OR. Register transfer operations allow data to move between registers efficiently during instruction execution.

RESULT

Thus, the register transfer system was designed and simulated successfully, and the ALU was used to perform addition, subtraction, AND, and OR operations successfully.

2. Single-Bus and Multi-Bus Organization

AIM

To create and compare single-bus and multi-bus computer organization models for performing data transfers between registers, memory, and I/O devices.

PROCEDURE

1. Design a single-bus organization containing CPU registers, memory, ALU, and I/O.
2. Connect the components using a common system bus.
3. Perform register-to-register data transfer.
4. Perform memory read and write operations.
5. Perform I/O data transfer.
6. Design a multi-bus organization using separate buses for different data transfers.
7. Perform the same operations using the multi-bus organization.
8. Record the number of transfer steps and compare the efficiency of both organizations.

INPUT

Component	Data
R1	20
R2	15
Memory	50
I/O	30

SINGLE-BUS OPERATIONS

R1 → Bus → R3

Memory → Bus → R4

I/O → Bus → R5

R1 → Bus → ALU → R6

In a single-bus system, only one major transfer can normally use the common bus at a time.

MULTI-BUS OPERATIONS

R1 → Bus A → R3

Memory → Bus B → R4

I/O → Bus C → R5

R1 and R2 → ALU → R6

Multiple transfers can occur simultaneously when different buses and resources are available.

COMPARISON

Parameter	Single-Bus	Multi-Bus
Number of buses	One	Multiple
Data transfer	Mostly sequential	Can be parallel
Bus contention	High	Low
Transfer time	Higher	Lower
Hardware complexity	Low	High

Parameter	Single-Bus	Multi-Bus
Cost	Lower	Higher
Performance	Moderate	High
Bottleneck	Common bus	Reduced

ANALYSIS

In a single-bus organization, all components share the same communication path. Therefore, simultaneous transfers are restricted and bus contention can occur. In a multi-bus organization, separate buses allow multiple data transfers to take place at the same time. This reduces waiting time and improves processor performance. Although the multi-bus system requires additional hardware and control circuitry, it provides better throughput and reduces bottlenecks.

RESULT

Thus, both single-bus and multi-bus organizations were modeled successfully. The multi-bus organization provided better parallel data transfer and higher efficiency than the single-bus organization.

3. Five-Stage Instruction Pipeline

AIM

To simulate a five-stage instruction pipeline consisting of Fetch, Decode, Execute, Memory, and Write-back stages and compare its performance with a non-pipelined processor.

PIPELINE STAGES

1. **IF – Instruction Fetch:** Fetches the instruction from memory.
2. **ID – Instruction Decode:** Decodes the instruction and identifies the operands.
3. **EX – Execute:** Performs the required ALU operation.
4. **MEM – Memory:** Performs memory read/write operations when required.
5. **WB – Write-back:** Stores the result in the destination register.

INPUT INSTRUCTIONS

I1: ADD R1, R2, R3

I2: SUB R4, R5, R6

I3: AND R7, R8, R9

I4: OR R10, R11, R12

PIPELINE EXECUTION

Clock Cycle	I1	I2	I3	I4
1	IF			

Clock Cycle	I1	I2	I3	I4
2	ID	IF		
3	EX	ID	IF	
4	MEM	EX	ID	IF
5	WB	MEM	EX	ID
6		WB	MEM	EX
7			WB	MEM
8				WB

For 4 instructions and 5 stages:

Pipeline clock cycles = k + n − 1

= 5 + 4 − 1

= **8 cycles**

A non-pipelined processor requires approximately:

5 × 4 = **20 cycles**

PERFORMANCE

Speedup:

Speedup = Non-pipelined execution time / Pipelined execution time

Speedup = 20 / 8

= **2.5**

Throughput of the pipeline increases because multiple instructions are processed simultaneously in different stages.

ANALYSIS

In a non-pipelined processor, an instruction must complete all five stages before the next instruction begins. In a pipelined processor, different instructions occupy different stages simultaneously. After the pipeline is filled, approximately one instruction can complete during each clock cycle. Therefore, pipelining improves instruction throughput and reduces the average execution time per instruction.

RESULT

Thus, the five-stage instruction pipeline was successfully simulated. The pipelined system required 8 cycles compared with approximately 20 cycles for the non-pipelined system, providing a speedup of approximately 2.5 for the given four-instruction example.

4. Pipeline Hazards and Hazard Resolution

AIM

To develop a pipeline execution scenario containing data, control, and structural hazards and implement stalling, data forwarding, and branch prediction techniques to reduce their effects.

TYPES OF HAZARDS

1. Data Hazard

A data hazard occurs when an instruction depends on the result of a previous instruction.

Example:

I1: ADD R1, R2, R3

I2: SUB R4, R1, R5

Here, I2 requires the value of R1 produced by I1.

Solution: Data forwarding can transfer the result directly from the execution stage of I1 to the execution stage of I2 without waiting for the write-back stage.

2. Control Hazard

A control hazard occurs when a branch instruction changes the normal sequence of instruction execution.

Example:

I1: ADD R1, R2, R3

I2: BEQ R1, R0, LABEL

I3: SUB R4, R5, R6

The processor may not immediately know whether I3 should be executed.

Solution: Branch prediction predicts whether the branch will be taken or not taken. Correct prediction avoids unnecessary pipeline stalls.

3. Structural Hazard

A structural hazard occurs when two pipeline stages require the same hardware resource simultaneously.

Example:

An instruction requires memory access while another instruction also requires the same memory unit.

Solution: Additional hardware resources or separate instruction and data memories can reduce structural hazards.

INPUT INSTRUCTION SEQUENCE

I1: LOAD R1, 1000H

I2: ADD R2, R1, R3

I3: SUB R4, R5, R6

I4: BEQ R2, R0, LABEL

I5: OR R7, R8, R9

HAZARD RESOLUTION

Hazard	Cause	Resolution
Data Hazard	I2 depends on I1	Data forwarding
Control Hazard	Branch instruction I4	Branch prediction
Structural Hazard	Shared hardware resource	Additional resources/stalling
Load-use dependency	Data required immediately after LOAD	One-cycle stall or forwarding

OBSERVATION

Without hazard handling:

- Instructions may produce incorrect results.
- Pipeline stalls increase.
- Processor throughput decreases.
- Branch instructions may cause incorrect instruction fetching.
- Shared resources can cause conflicts.

With hazard handling:

- Data forwarding reduces unnecessary stalls.
- Branch prediction reduces control-hazard penalties.
- Stalling prevents incorrect execution when forwarding is insufficient.
- Additional hardware resources reduce structural conflicts.

ANALYSIS

Pipeline hazards prevent ideal pipeline operation. Data hazards are mainly handled through forwarding and, when necessary, stalling. Control hazards can be reduced through branch prediction. Structural hazards can be minimized by providing sufficient hardware resources. These techniques allow the pipeline to continue operating efficiently while maintaining correct program execution.

RESULT

Thus, data, control, and structural hazards were successfully identified and analyzed. Stalling, data forwarding, and branch prediction techniques were applied to reduce pipeline delays and improve execution performance.

5. Superscalar, Out-of-Order, Speculative Execution and SMT

AIM

To analyze a superscalar processor using out-of-order and speculative execution and study hyper-threading/simultaneous multithreading to evaluate instruction throughput and processor resource utilization.

PROCEDURE

1. Design a processor with multiple execution units.
2. Fetch multiple instructions during each clock cycle.
3. Decode and issue independent instructions simultaneously.
4. Use out-of-order execution to execute ready instructions before instructions that are waiting for operands.
5. Use speculative execution to execute instructions before branch outcomes are completely known.
6. Use multiple hardware threads to allow instructions from different threads to share processor resources.
7. Compare the throughput and resource utilization with a single-issue processor.

INPUT INSTRUCTIONS

I1: ADD R1, R2, R3

I2: MUL R4, R5, R6

I3: SUB R7, R8, R9

I4: AND R10, R11, R12

SUPERSCALAR EXECUTION

Assume a **2-issue superscalar processor** with two available execution units.

Clock Cycle	**Execution Unit 1**	**Execution Unit 2**
1	I1	I2
2	I3	I4

Therefore, four independent instructions can be executed in approximately two issue cycles instead of four issue cycles in a single-issue processor.

OUT-OF-ORDER EXECUTION

Consider:

I1: LOAD R1, [1000H]

I2: ADD R4, R5, R6

I3: SUB R7, R1, R8

I3 depends on I1, but I2 is independent. Therefore, the processor can execute I2 while I1 is waiting for its data.

This prevents an independent instruction from unnecessarily waiting behind a dependent instruction.

SPECULATIVE EXECUTION

For a branch:

I1: BEQ R1, R2, LABEL

I2: ADD R3, R4, R5

The processor predicts the branch outcome and may execute I2 speculatively. If the prediction is correct, execution continues without a major delay. If the prediction is incorrect, the speculative instructions are discarded and the correct instructions are fetched.

HYPER-THREADING / SMT

Simultaneous Multithreading allows instructions from multiple threads to use processor resources during the same clock cycle.

Example:

Clock Cycle	**Thread 1**	**Thread 2**
1	ADD	MUL
2	SUB	AND
3	LOAD	OR

When one thread is waiting for memory or another resource, instructions from another thread can use otherwise idle execution resources.

PERFORMANCE COMPARISON

Feature	**Single-Issue Processor**	**Superscalar + SMT**
Instructions issued/cycle	1	Multiple
Execution order	Mostly sequential	Out-of-order possible
Branch handling	Limited	Speculative
Parallel execution	Low	High
Resource utilization	Moderate	Higher
Instruction throughput	Lower	Higher
Hardware complexity	Low	High

ANALYSIS

Superscalar processors improve performance by issuing multiple independent instructions in the same clock cycle. Out-of-order execution increases resource utilization by allowing ready instructions to execute without waiting for earlier instructions that are stalled. Speculative execution improves performance by predicting future control flow and executing instructions before branch results are known. Hyper-threading or SMT allows instructions from multiple threads to share execution resources, increasing processor utilization when one thread cannot fully use the available resources.

RESULT

Thus, superscalar, out-of-order, speculative execution, and SMT concepts were successfully analyzed. These techniques increase instruction-level parallelism, improve resource utilization, and increase instruction throughput compared with a conventional single-issue processor.

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- EXPERIMENTS

Course Code:	**CSA1223**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO3**	**Assessment:**	**ASSESSMENT TOOL 1- EXPERIMENTS**
Weightage:	**40%**	**Total Marks:**	**100**
CO3 Statement:	*Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)*		
Student Name:	G.N. Mogana Priya	**Reg.No.:**	192321165
Department:	IT	**Date:**	

79%

ANNEXURE A

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.
2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.
3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.
4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.
5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

Code of Conduct:

I G.N.Mogana Priya (192321165) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

MARKS ALLOCATION

	Understanding of Concepts (20%)	Experimental Design (30%)	Use of Tools (25%)	Analysis of Results (15%)	Documentation (10%)	
Q1	3	5	4	3	2	17
Q2	3	4	3	3	2	15
Q3	4	5	4	2	1	16
Q4	5	4	3	3	2	17
Q5	3	3	4	2	2	14

79

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies(5)	Good understanding with proper integration of most concepts(4)	Basic understanding; limited or partial integration(3)	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution(6)	Correct implementation with minor issues(4)	Basic implementation with errors or inefficiencies(3)	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration(4)	Appropriate use of tools with correct execution(3)	Limited or inconsistent use of tools(2)	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results(3)	Good analysis with reasonable interpretation(2)	Basic analysis with limited insights(1)	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results(2)	Organized documentation with necessary details(1.5)	Basic documentation with missing details(1)	Poor or incomplete documentation